

Below is a comprehensive Java Collections Framework reference covering the major **interfaces, abstract base classes, concrete implementations, legacy classes, and Map hierarchy**. The examples use modern Java syntax where appropriate.

1. Overall Collections Framework Hierarchy

Category	Main Interface / Class	Important Implementations
Root collection	Collection<E>	List, Set, Queue
List	List<E>	ArrayList, LinkedList, Vector, Stack
Set	Set<E>	HashSet, LinkedHashSet, SortedSet, NavigableSet
Sorted Set	SortedSet<E>	TreeSet
Navigable Set	NavigableSet<E>	TreeSet
Queue	Queue<E>	PriorityQueue, LinkedList, ArrayDeque
Deque	Deque<E>	ArrayDeque, LinkedList
Map root	Map<K,V>	HashMap, LinkedHashMap, TreeMap, Hashtable
Sorted Map	SortedMap<K,V>	TreeMap
Navigable Map	NavigableMap<K,V>	TreeMap
Concurrent Map	ConcurrentMap<K,V>	ConcurrentHashMap
Concurrent Navigable Map	ConcurrentNavigableMap<K,V>	ConcurrentSkipListMap
Legacy	Dictionary<K,V>	Hashtable
Legacy list	Vector<E>	Stack<E>
Abstract base	AbstractCollection<E>	Used as a base for custom collections
Abstract list	AbstractList<E>	Used by list implementations
Abstract sequential list	AbstractSequentialList<E>	Used by sequential-access lists
Abstract set	AbstractSet<E>	Base for set implementations

Abstract map	AbstractMap<K,V>	Base for map implementations
Abstract queue	AbstractQueue<E>	Base for queue implementations

2. Collection Framework Interfaces

Interface	Definition	Allows Duplicates?	Ordered?	Main Implementations	Example
Collection<E>	Root interface for most collection types containing groups of objects	Depends	Depends	ArrayList, HashSet, PriorityQueue	Collection<String> c = new ArrayList<>();
List<E>	Ordered collection where elements are accessed by index	Yes	Yes	ArrayList, LinkedList, Vector	List<String> l = new ArrayList<>();
Set<E>	Collection that does not allow duplicate elements	No	Depends	HashSet, LinkedHashSet, TreeSet	Set<Integer> s = new HashSet<>();
SortedSet<E>	Set maintaining elements according to their natural ordering or comparator	No	Sorted	TreeSet	SortedSet<Integer> s = new TreeSet<>();
NavigableSet<E>	Extended SortedSet supporting	No	Sorted	TreeSet	NavigableSet<Integer> s = new

	navigation operations such as lower, floor, ceiling and higher				TreeSet<>();
Queue<E>	Collection designed for holding elements before processing	Usually yes	Depends	PriorityQueue, LinkedList	Queue<String> q = new LinkedList<>();
Deque<E>	Double-ended queue supporting insertion/removal at both ends	Yes	Yes	ArrayDeque, LinkedList	Deque<String> d = new ArrayDeque<>();

Example

```
List<String> names = new ArrayList<>();
```

```
names.add("Anurag");
names.add("Rahul");
names.add("Anurag");
```

```
System.out.println(names);
```

Output:

```
[Anurag, Rahul, Anurag]
```

List allows duplicate values.

3. List Interface and Its Implementations

List<E>

List represents an **ordered collection**. Each element has an index beginning from 0.

Implementation	Internal	Duplicates	null	Random	Thread-	Typical Use
----------------	----------	------------	------	--------	---------	-------------

	Structure			Access	Safe	
ArrayList	Resizable array	Yes	Yes	Excellent	No	General-purpose list
LinkedList	Doubly linked list	Yes	Yes	Poor	No	Frequent insertion/removal
Vector	Resizable array	Yes	Yes	Good	Yes	Legacy synchronized list
Stack	Vector-based stack	Yes	Yes	Good	Yes	Legacy stack

ArrayList

ArrayList is a resizable-array implementation of List.

```
List<String> names = new ArrayList<>();
```

```
names.add("Anurag");
names.add("Rahul");
names.add("Amit");
```

```
System.out.println(names.get(1));
```

Output:

Rahul

Best when: you frequently access elements by index.

LinkedList

LinkedList implements both List and Deque.

```
LinkedList<String> names = new LinkedList<>();
```

```
names.add("Anurag");
names.addFirst("Rahul");
names.addLast("Amit");
```

```
System.out.println(names);
```

Output:

[Rahul, Anurag, Amit]

Best when: you need list operations together with queue/deque operations.

Vector

Vector is a legacy synchronized resizable-array implementation.

```
Vector<Integer> numbers = new Vector<>();
```

```
numbers.add(10);  
numbers.add(20);  
numbers.add(30);
```

```
System.out.println(numbers);
```

Generally, prefer ArrayList for non-concurrent code.

Stack

Stack extends Vector and represents a legacy LIFO stack.

```
Stack<Integer> stack = new Stack<>();
```

```
stack.push(10);  
stack.push(20);  
stack.push(30);
```

```
System.out.println(stack.pop());
```

Output:

30

For new code, ArrayDeque is generally preferred for stack behavior.

4. Set Interface and Implementations

A Set does **not permit duplicate elements**.

Implementation	Ordering	Duplicates	null	Performance Characteristics	Typical Use
HashSet	No guaranteed order	No	One null allowed	Fast average lookup	General-purpose unique values
LinkedHashSet	Insertion order	No	One null allowed	Slightly more overhead than HashSet	Unique values + insertion order
TreeSet	Sorted order	No	Generally no null with natural ordering	$O(\log n)$ basic operations	Sorted unique values

HashSet

Uses hashing to store unique elements.

```
Set<Integer> numbers = new HashSet<>();
```

```
numbers.add(10);
numbers.add(20);
numbers.add(10);
```

```
System.out.println(numbers);
```

The second 10 is ignored.

Important characteristics

Property	HashSet
Duplicate values	Not allowed
Ordering	No guaranteed iteration order
Lookup	Average $O(1)$
Add	Average $O(1)$
Remove	Average $O(1)$
Data structure concept	Hash table

--	--

LinkedHashSet

Maintains insertion order.

```
Set<String> names = new LinkedHashSet<>();
```

```
names.add("Rahul");  
names.add("Anurag");  
names.add("Amit");
```

```
System.out.println(names);
```

Output:

[Rahul, Anurag, Amit]

TreeSet

Stores unique elements in sorted order.

```
Set<Integer> numbers = new TreeSet<>();
```

```
numbers.add(50);  
numbers.add(10);  
numbers.add(30);
```

```
System.out.println(numbers);
```

Output:

[10, 30, 50]

TreeSet implements NavigableSet.

5. SortedSet and NavigableSet

Interface	Purpose	Important Methods	Implementation
SortedSet<E>	Represents a set sorted according to natural ordering or a	first(), last(), headSet(), tailSet(), subSet()	TreeSet

	comparator		
NavigableSet<E>	Extends SortedSet with navigation operations	lower(), floor(), ceiling(), higher(), pollFirst(), pollLast()	TreeSet

Example:

```
NavigableSet<Integer> numbers = new TreeSet<>();
```

```
numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);
```

```
System.out.println(numbers.lower(30)); // 20
System.out.println(numbers.floor(30)); // 30
System.out.println(numbers.ceiling(25)); // 30
System.out.println(numbers.higher(30)); // 40
```

6. Queue Interface

Queue<E> represents a collection designed to hold elements before processing.

Implementation	Ordering	Main Concept	Typical Use
LinkedList	FIFO/deque	Linked nodes	General queue
PriorityQueue	Priority-based	Heap	Priority processing
ArrayDeque	FIFO/deque	Resizable array	High-performance queue/deque

Example:

```
Queue<String> queue = new LinkedList<>();
```

```
queue.offer("A");
queue.offer("B");
queue.offer("C");
```

```
System.out.println(queue.poll());
```

Output:

7. PriorityQueue

PriorityQueue processes elements according to priority rather than insertion order.

```
PriorityQueue<Integer> queue = new PriorityQueue<>();
```

```
queue.offer(50);  
queue.offer(10);  
queue.offer(30);
```

```
System.out.println(queue.poll());
```

Output:

10

The smallest element has the highest priority when natural ordering is used.

8. Deque Interface

Deque means **Double Ended Queue**.

It allows insertion and removal from both the front and back.

Method	Beginning	End
Insert	addFirst()	addLast()
Insert safely	offerFirst()	offerLast()
Remove	removeFirst()	removeLast()
Remove safely	pollFirst()	pollLast()
Examine	getFirst()	getLast()
Examine safely	peekFirst()	peekLast()

Example:

```
Deque<Integer> deque = new ArrayDeque<>();
```

```
deque.addFirst(20);  
deque.addLast(30);  
deque.addFirst(10);
```

```
System.out.println(deque);
```

Output:

```
[10, 20, 30]
```

9. Map Framework

An important distinction:

Map is part of the Java Collections Framework, but Map does not extend Collection.

A Map stores data as **key-value pairs**.

Key → Value

Example:

```
101 → "Anurag"  
102 → "Rahul"  
103 → "Amit"
```

10. Map Interfaces

Interface	Definition	Duplicate Keys?	Ordering	Important Implementation
Map<K,V>	Stores key-value mappings	No	Depends on implementation	HashMap
SortedMap<K, V>	Map whose keys are sorted	No	Sorted by key	TreeMap
NavigableMap<K,V>	Extends SortedMap with navigation operations	No	Sorted	TreeMap
ConcurrentMap<K,V>	Map designed for concurrent access	No	Depends	ConcurrentHashMap

ConcurrentNavigableMap<K, V>	Concurrent + navigable sorted map	No	Sorted	ConcurrentSkipListMap
---	-----------------------------------	----	--------	-----------------------

11. HashMap

HashMap is the most commonly used general-purpose map.

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");
employees.put(102, "Rahul");
employees.put(103, "Amit");
```

```
System.out.println(employees.get(101));
```

Output:

Anurag

Characteristics

Property	HashMap
Key-value pairs	Yes
Duplicate keys	No
Duplicate values	Yes
Ordering	No guaranteed order
null key	One
null values	Multiple
Average lookup	O(1)
Thread-safe	No

12. LinkedHashMap

LinkedHashMap maintains a predictable iteration order, normally insertion order.

```
Map<Integer, String> employees = new LinkedHashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");
```

```
System.out.println(employees);
```

Output preserves insertion order.

It can also be configured for **access-order**, which makes it useful for LRU-cache-style implementations.

13. TreeMap

TreeMap maintains keys in sorted order.

```
Map<Integer, String> employees = new TreeMap<>();
```

```
employees.put(103, "Amit");  
employees.put(101, "Anurag");  
employees.put(102, "Rahul");
```

```
System.out.println(employees);
```

Conceptually:

```
101 → Anurag  
102 → Rahul  
103 → Amit
```

Characteristics

Property	TreeMap
Keys	Sorted
Duplicate keys	No
Duplicate values	Yes
null key	Generally not supported with natural ordering
Basic operations	$O(\log n)$
Interface	NavigableMap

Internal concept	Red-black tree
-------------------------	----------------

14. Hashtable

Hashtable is a legacy synchronized map implementation.

```
Hashtable<Integer, String> table = new Hashtable<>();
```

```
table.put(101, "Anurag");  
table.put(102, "Rahul");
```

```
System.out.println(table.get(101));
```

Important:

Hashtable does not allow null keys or null values.

For modern concurrent applications, ConcurrentHashMap is usually preferred.

15. ConcurrentHashMap

ConcurrentHashMap is designed for concurrent access by multiple threads.

```
Map<Integer, String> employees =  
    new ConcurrentHashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");
```

```
System.out.println(employees.get(101));
```

It is commonly used when multiple threads need to read and update a map concurrently.

16. ConcurrentSkipListMap

ConcurrentSkipListMap is a concurrent, sorted, navigable map.

```
ConcurrentNavigableMap<Integer, String> map =  
    new ConcurrentSkipListMap<>();
```

```
map.put(103, "Amit");  
map.put(101, "Anurag");
```

```
map.put(102, "Rahul");
```

```
System.out.println(map);
```

Keys remain sorted while supporting concurrent access.

17. WeakHashMap

WeakHashMap uses weak references for its keys.

```
Map<Object, String> map = new WeakHashMap<>();
```

```
Object key = new Object();
```

```
map.put(key, "Some Value");
```

If the key is no longer strongly referenced elsewhere, the garbage collector may remove the corresponding entry.

Typical use: caches or mappings where entries should not prevent otherwise-unused keys from being garbage-collected.

18. IdentityHashMap

IdentityHashMap compares keys using **reference identity (==)** rather than normal equals() semantics.

```
Map<String, String> map = new IdentityHashMap<>();
```

```
String a = new String("Java");
```

```
String b = new String("Java");
```

```
map.put(a, "A");
```

```
map.put(b, "B");
```

```
System.out.println(map.size());
```

Output:

2

because:

a != b

even though:

```
a.equals(b) == true
```

19. EnumMap

EnumMap is a specialized map whose keys must be enum constants.

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY  
}
```

```
EnumMap<Day, String> schedule =  
    new EnumMap<>(Day.class);
```

```
schedule.put(Day.MONDAY, "Java");  
schedule.put(Day.TUESDAY, "SQL");
```

It is highly efficient when enum keys are used.

20. Abstract Collection Classes

Java provides abstract classes that make it easier to create custom collection implementations.

Abstract Class	Implements / Extends	Purpose
AbstractCollection<E>	Collection<E>	Skeletal implementation for collections
AbstractList<E>	AbstractCollection<E>, List<E>	Skeletal implementation for lists
AbstractSequentialList<E>	AbstractList<E>	Base for sequential-access lists
AbstractSet<E>	AbstractCollection<E>, Set<E>	Skeletal implementation for sets
AbstractQueue<E>	AbstractCollection<E>, Queue<E>	Skeletal implementation for queues
AbstractMap<K,V>	Map<K,V>	Skeletal implementation for maps
AbstractMap.SimpleEntry<K,V>	Entry implementation	Mutable key-value entry

AbstractMap.SimpleImmutableEntry<K,V>	Entry implementation	Immutable key-value entry
--	----------------------	---------------------------

21. Important Concrete Collection Classes

Class	Interface(s)	Main Purpose
ArrayList	List, RandomAccess	Dynamic array
LinkedList	List, Deque	Linked list + deque
Vector	List	Legacy synchronized list
Stack	List through Vector	Legacy LIFO stack
HashSet	Set	Hash-based unique collection
LinkedHashSet	Set	Unique collection with insertion order
TreeSet	NavigableSet	Sorted unique collection
PriorityQueue	Queue	Priority-based processing
ArrayDeque	Deque	High-performance deque
HashMap	Map	General-purpose hash map
LinkedHashMap	Map	Ordered hash map
TreeMap	NavigableMap	Sorted map
Hashtable	Map	Legacy synchronized map
WeakHashMap	Map	Weak-key map
IdentityHashMap	Map	Identity-based key comparison
EnumMap	Map	Enum-key map
ConcurrentHashMap	ConcurrentMap	Concurrent hash map
ConcurrentSkipListMap	ConcurrentNavigableMap	Concurrent sorted map
CopyOnWriteArrayList	List	Thread-safe copy-on-write list
CopyOnWriteArraySet	Set	Thread-safe copy-on-write set

ConcurrentLinkedQueue	Queue	Non-blocking concurrent queue
ConcurrentLinkedDeque	Deque	Non-blocking concurrent deque
ArrayBlockingQueue	BlockingQueue	Bounded blocking queue
LinkedBlockingQueue	BlockingQueue	Blocking FIFO queue
PriorityBlockingQueue	BlockingQueue	Blocking priority queue
DelayQueue	BlockingQueue	Delayed-element queue
SynchronousQueue	BlockingQueue	Direct producer-consumer handoff
LinkedTransferQueue	TransferQueue	Advanced producer-consumer queue
ConcurrentSkipListSet	NavigableSet	Concurrent sorted set

22. Concurrent Collection Interfaces

The concurrent collections are particularly important in multithreaded applications.

Interface	Definition	Implementation
ConcurrentMap<K,V>	Map supporting concurrent operations	ConcurrentHashMap
ConcurrentNavigableMap<K,V>	Concurrent sorted/navigable map	ConcurrentSkipListMap
BlockingQueue<E>	Queue supporting blocking insertion/removal	ArrayBlockingQueue, LinkedBlockingQueue
BlockingDeque<E>	Blocking double-ended queue	LinkedBlockingDeque
TransferQueue<E>	Queue supporting producer-to-consumer transfer	LinkedTransferQueue

Example:

```
BlockingQueue<String> queue =
    new ArrayBlockingQueue<>(10);
```

```
queue.put("Java");

String value = queue.take();

System.out.println(value);
```

23. CopyOnWrite Collections

Class	Purpose	Best Use
CopyOnWriteArrayList	Thread-safe list where modifications create a new underlying array	Many reads, very few writes
CopyOnWriteArraySet	Set based on copy-on-write semantics	Many reads, few modifications

Example:

```
List<String> names =
    new CopyOnWriteArrayList<>();

names.add("Anurag");
names.add("Rahul");
```

A key characteristic is that iteration is designed to work safely while other threads modify the collection.

24. Full Hierarchy — Simplified View

```
Iterable
|
+-- Collection
|   |
|   +-- List
|       |
|       +-- ArrayList
|       +-- LinkedList
|       +-- Vector
|           |
|           +-- Stack
|
+-- Set
|   |
|   +-- HashSet
|       |
|       |
```

```

|      |      +-- HashSet
|      |      |
|      |      +-- SortedSet
|      |      |
|      |      +-- NavigableSet
|      |      |
|      |      +-- TreeSet
|
+-- Queue
|
+-- PriorityQueue
+-- Deque
|
+-- ArrayDeque
+-- LinkedList

```

The Map hierarchy is separate:

```

Map
|
+-- SortedMap
|   |
|   +-- NavigableMap
|   |
|   +-- TreeMap
|
+-- HashMap
|   |
|   +-- LinkedHashMap
|
+-- Hashtable
|
+-- WeakHashMap
+-- IdentityHashMap
+-- EnumMap
|
+-- ConcurrentMap
|   |
|   +-- ConcurrentHashMap
|   |
|   +-- ConcurrentNavigableMap
|       |
|       +-- ConcurrentSkipListMap

```

25. Collection vs Map

Feature	Collection	Map
Main purpose	Store individual elements	Store key-value mappings
Data representation	E	K,V

Duplicate keys	Not applicable	Not allowed
Duplicate values	Depends on collection	Allowed
Access	Element-based	Key-based
Root interface	Collection	Map
Example	List<String>	Map<Integer,String>
Common implementation	ArrayList	HashMap

Example:

```
List<String> list = new ArrayList<>();
```

```
list.add("Java");
list.add("Spring");
```

versus:

```
Map<Integer, String> map = new HashMap<>();
```

```
map.put(101, "Java");
map.put(102, "Spring");
```

26. Quick Selection Guide

Requirement	Recommended Class
General-purpose list	ArrayList
Frequent indexed access	ArrayList
List + deque operations	LinkedList
Unique elements, no ordering requirement	HashSet
Unique elements + insertion order	LinkedHashSet
Unique elements + sorted order	TreeSet
FIFO queue	ArrayDeque
Priority-based processing	PriorityQueue

Stack/LIFO	ArrayDeque
General-purpose key-value storage	HashMap
Map + insertion order	LinkedHashMap
Map + sorted keys	TreeMap
Enum keys	EnumMap
Weakly referenced keys	WeakHashMap
Identity (==) key comparison	IdentityHashMap
Concurrent key-value map	ConcurrentHashMap
Concurrent sorted map	ConcurrentSkipListMap
Thread-safe list with many reads/few writes	CopyOnWriteArrayList
Thread-safe set with many reads/few writes	CopyOnWriteArraySet
Blocking producer-consumer queue	BlockingQueue implementation
Bounded blocking queue	ArrayBlockingQueue
Linked blocking queue	LinkedBlockingQueue

27. Most Important Interfaces to Remember

For Java interviews and real-world development, this hierarchy is especially important:

Level	Interface	What to Remember
1	Iterable	Supports enhanced for loop
2	Collection	Root of List/Set/Queue families
3	List	Ordered + duplicates
3	Set	Unique elements
3	Queue	Processing-oriented collection
4	SortedSet	Sorted unique elements

4	NavigableSet	Sorted + navigation
4	Deque	Both ends
Separate	Map	Key-value pairs
2	SortedMap	Sorted keys
3	NavigableMap	Sorted keys + navigation
Concurrent	ConcurrentMap	Thread-safe map operations
Concurrent	ConcurrentNavigableMap	Concurrent sorted map

Key interview point

The most important conceptual distinction is:

```
Collection
├── List
├── Set
├── Queue
│   └── Deque
```

```
Map ← separate hierarchy
├── SortedMap
│   └── NavigableMap
├── ConcurrentMap
│   └── ConcurrentNavigableMap
```

So, **Map is not a subtype of Collection**, even though both are major parts of the Java Collections Framework.